MODULE 3

3	**Relational Database Design**	
	3.1	Design guidelines for relational schemas, Functional Dependencies, Armstrong Axiom
	3.2	First Normal form, Second Normal form, Third normal Form.
	3.3	Decomposition and Desirable properties of decomposition, decomposition using multivalued dependencies, fourth normal form.
	3.4	Overall database design

Introduction to database design

- The design of a relational schema decides its goodness.
- Two approaches for database design :

a) ***Bottom up design methodology*** (Design by synthesis) : Relationships among attributes are considered as the starting point and using this to construct schemas.

b) ***Top down design methodology*** (design by analysis): starts with number of grouping of attributes to relations.

- The implicit goals of database design is to ***reduce redundancy*** *and* ***preserve information***

Informal guidelines to design relational schemas

- The four informal guidelines to design a relational schema are :

a) Semantics of the attributes must be clear in the schema

b) Reducing redundant information in tuples

c) Reducing the null values in tuples

d) Disallowing the possibility of generating spurious tuples.

1. Imparting clear semantics to attributes in a relation

- When we group attributes to form a relation, the attributes must have a real world meaning and proper interpretation associated with them.
- Consider the schema below:

EMPLOYEE

Ename	ssn	bdate	address	dnumber

- The easier to explain the semantics of the relation, the better the relation schema design will be.

EMPLOYEE-DETAILS

Ename	ssn	bdate	address	dnumber	Dpjct_nos

Guideline 1

- ***Design a relational schema so that it is easy to explain its meaning. Do not combine attributes from multiple entity types and relationship types in to a single relation.***
- Violating guideline 1:

EMPLOYEE-DEPT

Ename	Ssn	Bdate	Address	Dnumber	Dname	Dmgr-ssn

- This combines the attributes of two real world entities EMPLOYEE and DEPARTMENT, which violates guideline 1.

2. Redundant information in tuples

- Main goal of a schema design is **to minimize storage space** used by base relations.
- Consider the relation schema below:

EMPLOYEE-DEPT

Ename	Ssn	Bdate	Address	Dnumber	Dname	Dmgr-ssn

EMPLOYEE

Ename	ssn	bdate	address	dnumber

DEPARTMENT

Dname	dnumber	Dmgr-ssn

The table EMPLOYEE-DEPT is the result of join of EMPLOYEE and DEPARTMENT.

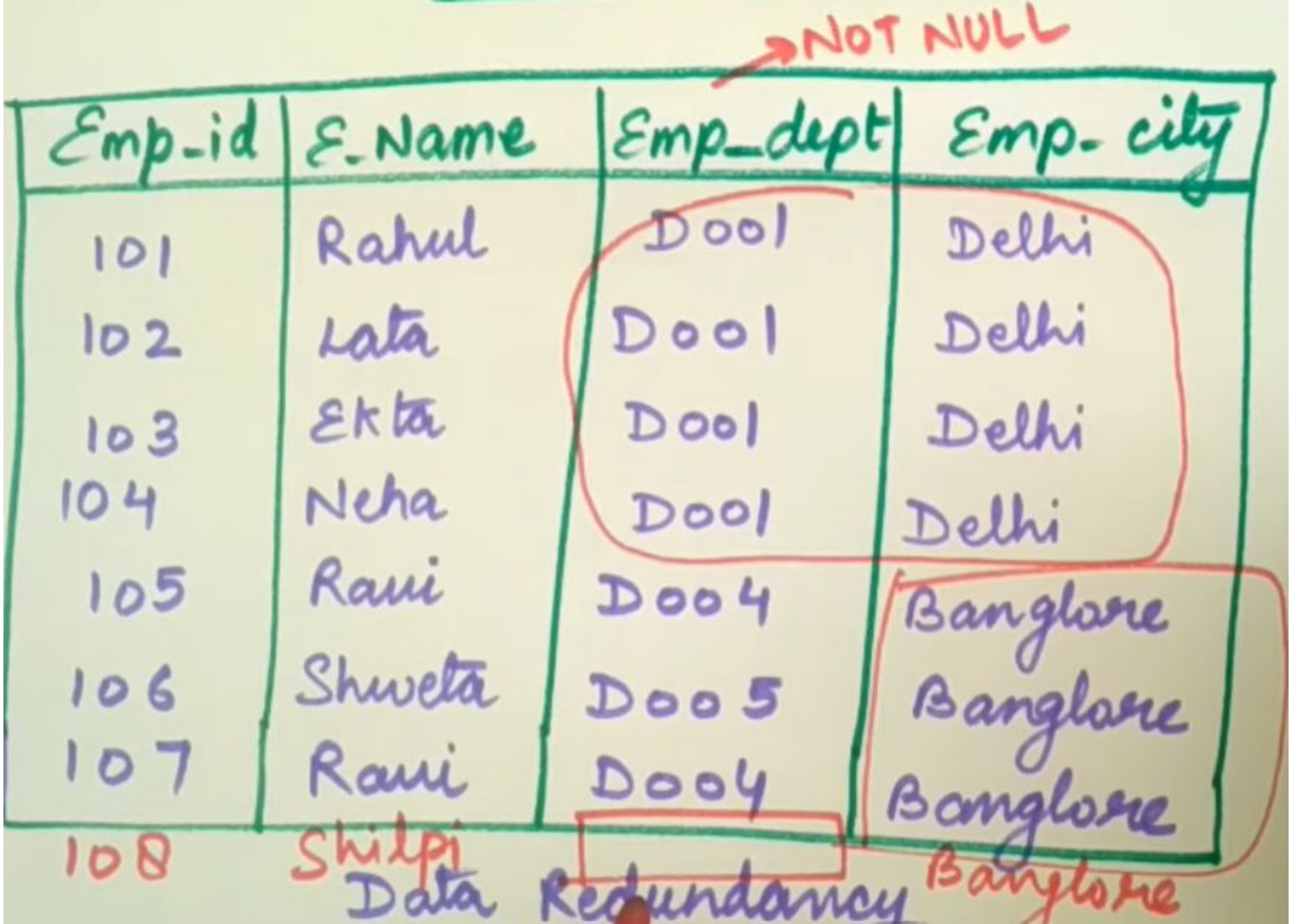
- The natural joins of base relations leads to additional problems referred as update anomalies : insertion anomalies, deletion anomalies, update anomalies.

- Insertion anomaly :

EMPLOYEE-DEPT

Ename	Ssn	Bdate	Address	Dnumber	Dname	Dmgr-ssn

1) Here to enter the details of an employee, details of his department lso must be entered. It results in repetition of details for each employee in the same department.
2) It is difficult to enter the details of a department with no employee, it results in null values for attributes


NOT NULL
Emp_id
E_Name
Emp_dept
Emp_city
101
Rahul
D001
Delhi
102
Lata
D001
Delhi
103
Ekta
D001
Delhi
104
Neha
D001
Delhi
105
Ravi
D004
Banglore
106
Shweta
D005
Banglore
107
Ravi
D004
Banglore
108
Shilpi
Banglore
Data Redundancy

- Deletion anomaly :

EMPLOYEE-DEPT

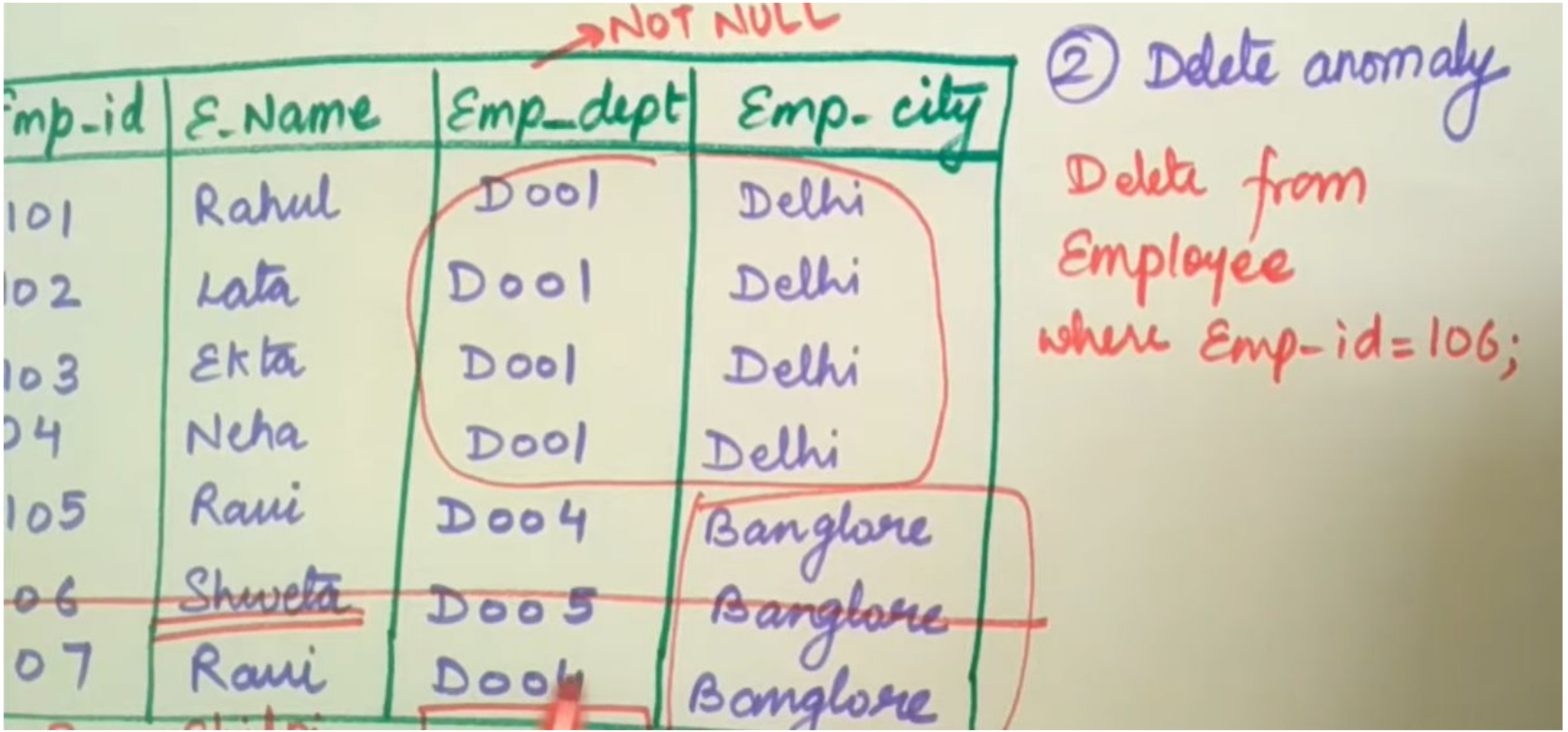
Ename	Ssn	Bdate	Address	Dnumber	Dname	Dmgr-ssn

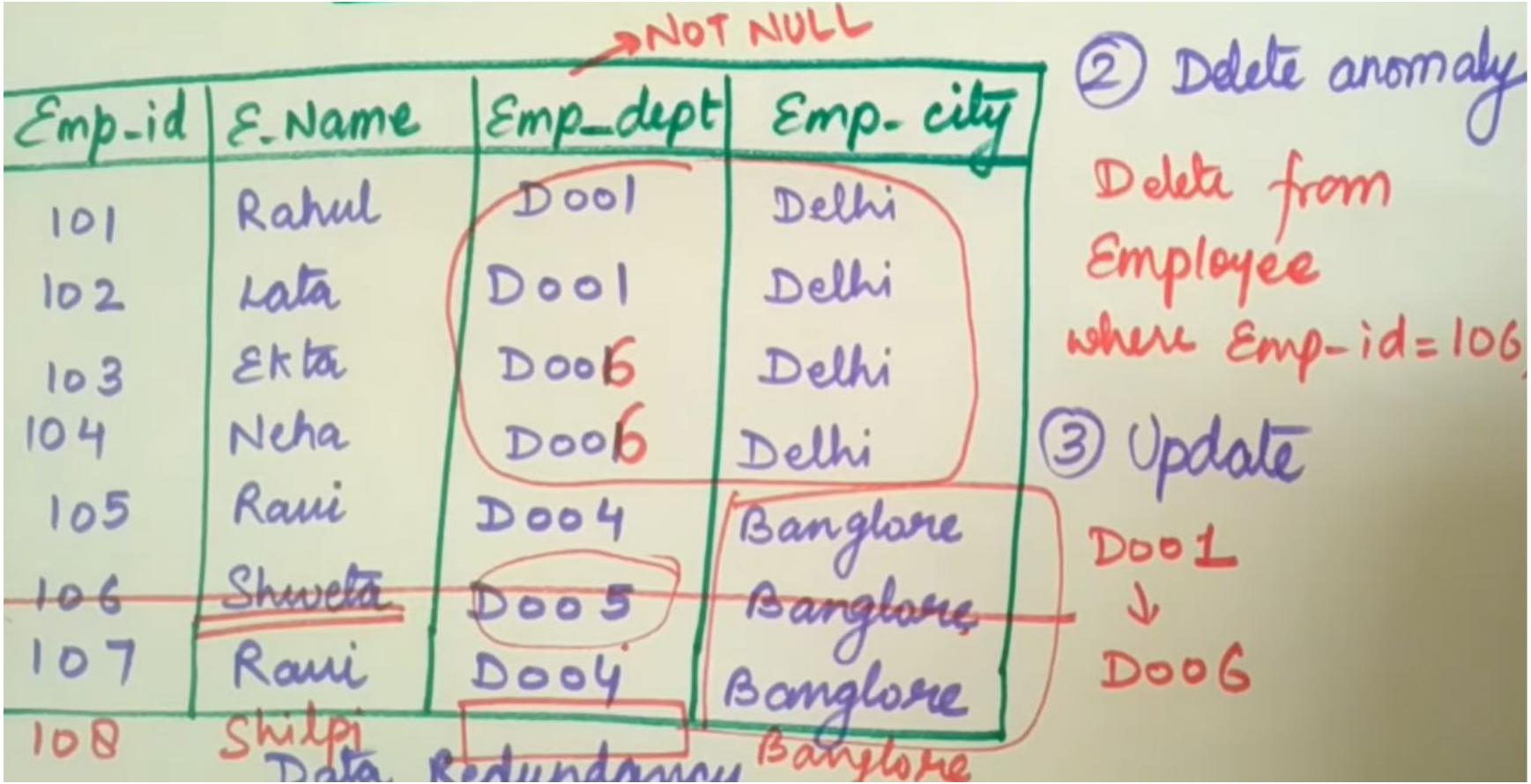
If the details of the last employee of a department is deleted, then the details of that department also no longer exist in the database.

- Modification anomaly :

In the above schema, if the value one attribute (eg: mgrssn of a department) is changed, that value should be updated for all the employee tuples in the same department.

Guideline 2 : ***Design a base relation schema so that no insertion, deletion or modification anomalies present in the relation. If any anomaly exists, make sure that the programs that update the database operate correctly.***


NOT NULL
Emp_id
E_Name
Emp_dept
Emp_city
101
Rahul
D001
Delhi
102
Lata
D001
Delhi
103
Ekta
D001
Delhi
04
Neha
D001
Delhi
105
Ravi
D004
Banglore
06
Shweta
D005
Banglore
07
Ravi
D004
Banglore
② Delete anomaly
Delete from
Employee
where Emp_id=106;


NOT NULL
Emp_id
E_Name
Emp_dept
Emp_city
101 Rahul D001 Delhi
102 Lata D001 Delhi
103 Ekta D006 Delhi
104 Neha D006 Delhi
105 Ravi D004 Banglore
106 Shweta D005 Banglore
107 Ravi D004 Banglore
108 Shilpi Banglore
Data Redundancy
② Ddete anomaly
Delete from Employee where Emp_id=106
③ Update
D001 ↓ D006

3. NULL values in tuples

- Null values lead to problems with understanding the meaning of the attributes.
- Also it causes confusions in COUNT and SUM operations.
- Also if NULL value comes in comparison in SELECT or JOIN operations, the result will be unpredictable.

Guideline 3:

As far as possible, avoid placing attributes in a relation whose values may frequently NULL. If NULLs are unavoidable make sure that they do not apply to majority of the tuples.

4. Generation of spurious tuples

- EMP-PROJ

Ssn	Pnumber	Hours	Ename	Pname	Plocation

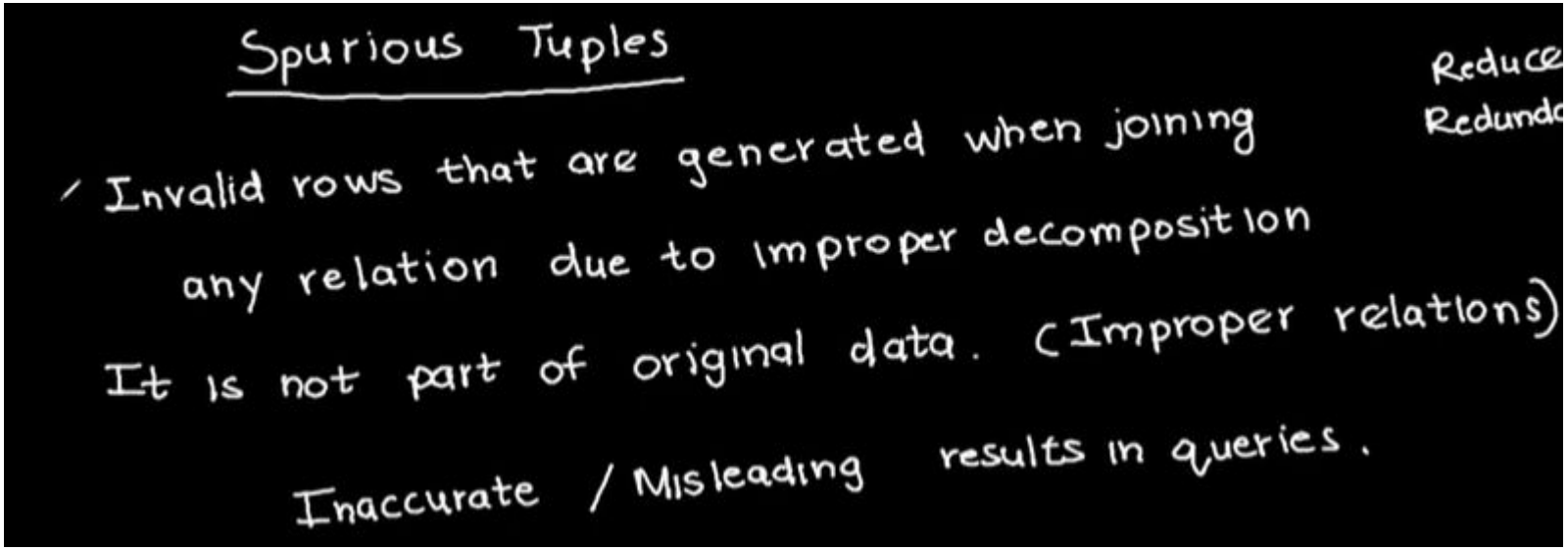
- EMP-LOCS

Ename	Plocation

- EMP-PROJ1

Ssn	Pnumber	Hours	Pname	Plocation

The relation EMP-PROJ is decomposed in to two relations EMP-LOCS and EMP-PROJ1. But if the two tables are joined by JOIN operation, it will generate some spurious tuples. So the joined table will not be same as EMP-PROJ.

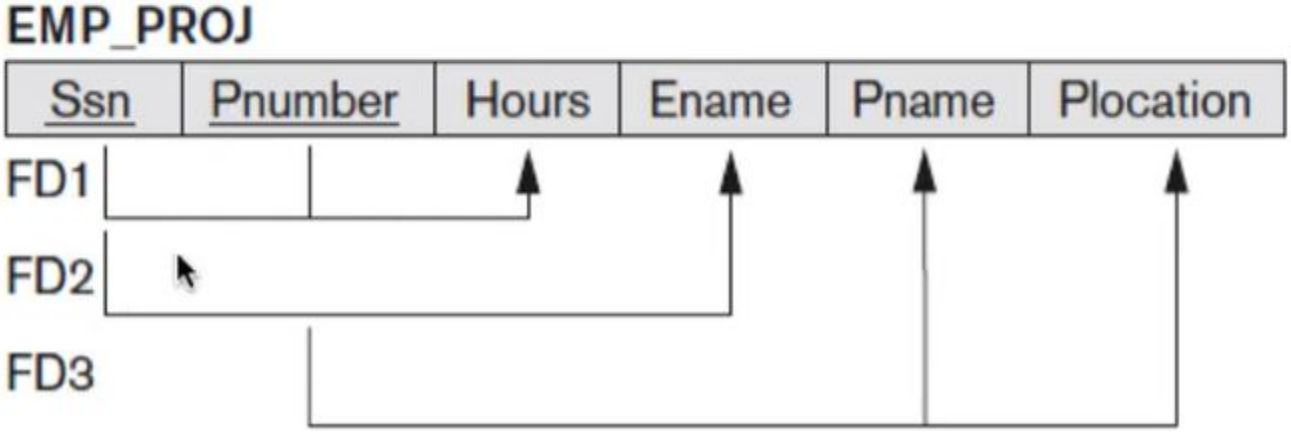


- **Guideline 4:**

Design relation schema so that they can be joined with equality conditions on attributes and they are appropriately related (primary key, foreign key) pairs in a way it guarantee that no spurious tuples will be generated.

FUNCTIONAL DEPENDENCY

A functional dependency is a constraint between two sets of attributes from the database.



A **functional dependency**, denoted by

X ->*Y*, between two sets of attributes *X* and *Y* that are subsets of *R* specifies a *constraint* on the possible tuples that can form a relation state *r* of *R*.

The constraint is that, for any two tuples *t*1 and *t*2 in *r* that have *t*1[*X*] = *t*2[*X*], they must also have

*t*1[*Y*] = *t*2[*Y*].

R.No → R.No ✓

(R.No, Name) → Name

* Relationship b/w one Attribute with another in a table

X → Y is FD if it satisfies a Contraint

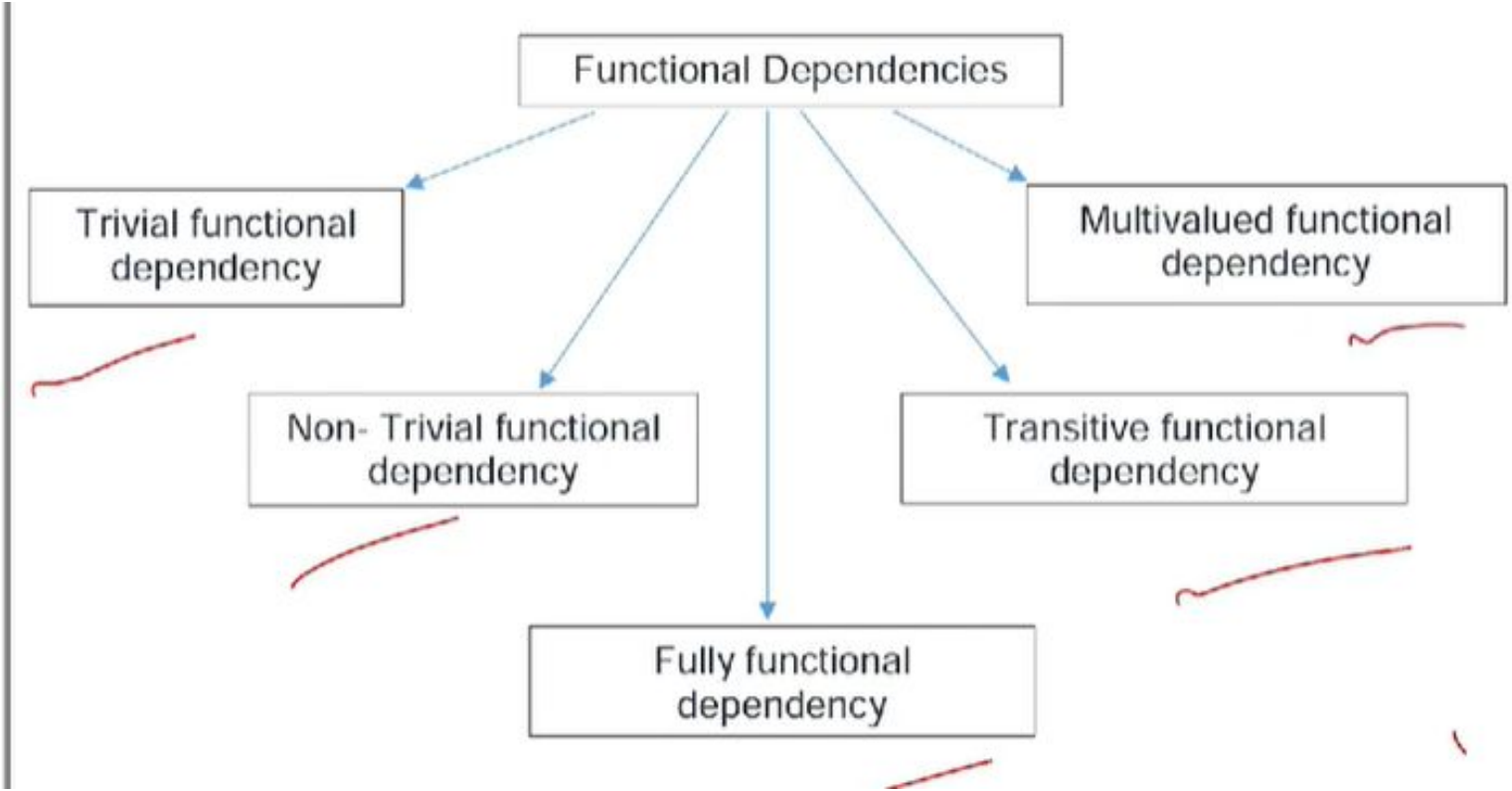
If $t_1.X = t_2.X$ then $t_1.Y = t_2.Y$

	X	Y
t_1	10	5
	20	6
	30	7
t_2	10	8

X=20 → Y=6

X=10 → Y=5/8

[Types of Functional Dependencies | Functional Dependency | DBMS]



1. Trivial Functional Dependency

In **Trivial Functional Dependency**, a dependent is always a subset of the determinant. i.e. If **X → Y** and **Y is the subset of X**, then it is called trivial functional dependency

Example:

roll_no	name	age
42	abc	17
43	pqr	18
44	xyz	18

R.No → R.No ✓
(R.No, Name) → Name

Here, {roll_no, name} → name is a trivial functional dependency, since the dependent name is a subset of determinant set {roll_no, name}. Similarly, roll_no → roll_no is also an example of trivial functional dependency.

Types of Functional Dependencies | Functional Dependency | DBMS

Non-trivial Functional Dependency

In Non-trivial functional dependency, the dependent is strictly not a subset of the determinant. i.e. If **X** → **Y** and **Y is not a subset of X**, then it is called Non-trivial functional dependency.

roll_no	name	age
42	abc	17
43	pqr	18
44	xyz	18

roll → name

name → age

roll, name →

Here, **roll_no → name** is a non-trivial functional dependency, since the dependent **name** is **not a subset of** determinant **roll_no**. Similarly, **{roll_no, name} → age** is also a non-trivial functional dependency, since **age** is **not a subset of {roll_no, name}**

3. Multivalued Functional Dependency

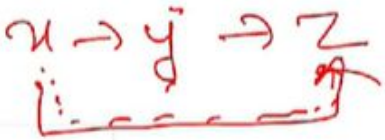
In Multivalued functional dependency, entities of the dependent set are **not dependent on each other**. i.e. If **a → {b, c}** and there exists **no** functional dependency between **b and c**, then it is called a **multivalued** functional dependency.

roll_no	name	age
42	abc	17
43	pqr	18
44	xyz	18
45	abc	19

Here, **roll_no → {name, age}** is a multivalued functional dependency, since the dependents **name** & **age** are **not dependent** on each other(i.e. **name → age** or **age → name doesn't exist** !)

4. Transitive Functional Dependency

In transitive functional dependency, dependent is indirectly dependent on determinant. i.e. If **a** → **b** & **b** → **c**, then according to axiom of transitivity, **a** → **c**. This is a **transitive functional dependency.**



For example,

enrol_no	name	dept	building_no
42	abc	CO	4
43	pqr	EC	2
44	xyz	IT	1
45	abc	EC	2

Here, **enrol_no** → **dept** and **dept** → **building_no.** Hence, according to the axiom of transitivity, **enrol_no** → **building_no** is a valid functional dependency. This is an indirect functional dependency, hence called Transitive functional dependency.

Full Functional Dependency :

- **Definition :**

  A Functional Dependency A → B is a full functional dependency if removal of any attributes from A means that the dependency does not hold any more.

- **Example :**

  {Emp_no, Project_no} → HOURS

  i.e. Emp_no → Hours

  and Project_no → Hours

- In the above example, Hours is fully functionally dependent on both Emp_no and Project_no.
- The number of hours spent on the project by a particular employee cannot be determined with the project number (Project_no) alone. It needs the employee number (Emp_no) as well.
- **Example :**

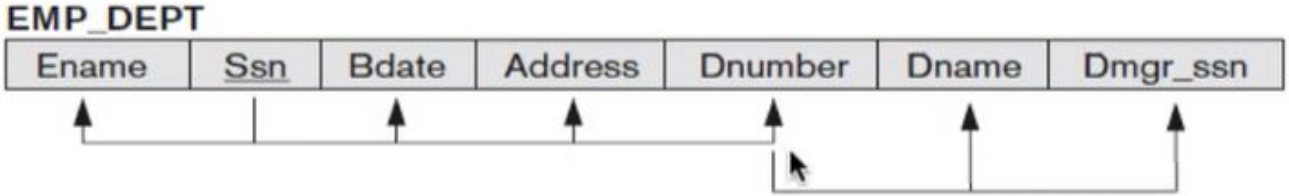
  If one of the attribute is not present (Null) then relations does not holds true.

Emp_no	Project_no	HOURS
1	A1	12

We denote by *F* the set of functional dependencies that are specified on relation schema *R*, dependencies can be *inferred* or *deduced* from the FDs in *F*.

define a concept called ***closure*** formally that includes all possible dependencies that can be inferred from the given set *F*.

Definition of Closure. Formally, the set of all dependencies that include F as well as all dependencies that can be inferred from F is called the **closure** of F; it is denoted by F^+.



Example: for the above example

F = {Ssn -> {Ename, Bdate, Address, Dnumber},

Dnumber ->{Dname, Dmgr_ssn} }

Some of the additional functional dependencies that we can *infer* from F are the following:

Ssn -> {Dname, Dmgr_ssn}
Ssn -> Ssn
Dnumber -> Dname

Armstrong Axioms

To determine a systematic way to infer dependencies, there are a set of **inference rules** that can be used to infer new dependencies from a given set of dependencies

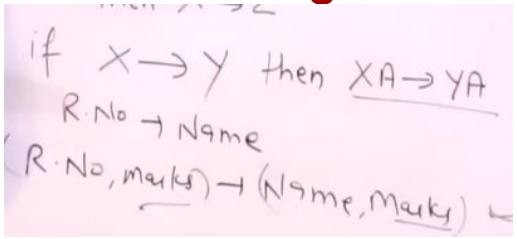
Eg:-- Dname is subset of Dnum, Dnum gives Dname

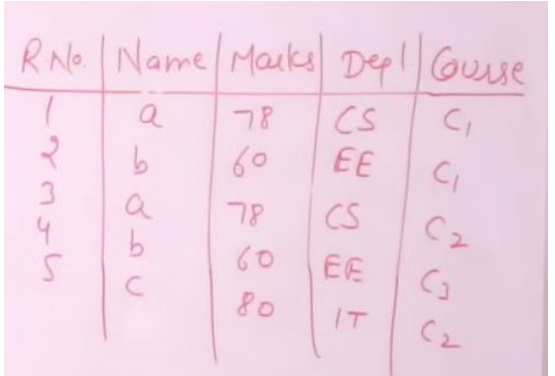
IR1. (Reflexive rule) If **Y** is a *subset-of* **X** (**Y** $\subseteq$ **X**), then **X** → **Y**

[states that a set of attributes determines itself or any of its subsets. Since IR1 generates dependencies that are always true, such dependencies are called Trivial otherwise it is called Non-trivial]. Formally, a functional dependency *X->Y* is **trivial** if *X* $\subseteq$ *Y*; otherwise, it is **nontrivial**

Eg:- Ssn gives Ename belongs to IR1
Ssn gives Bdate

IR2. (Augmentation rule)
If **X** → **Y**, then **XZ** → **YZ**





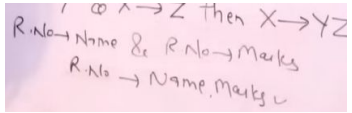
IR3. (Transitive rule)
If **X** → **Y** and **Y** → **Z**, then **X** → **Z**

IR4. (Decomposition/projective rule)

If **X → YZ**, then **X → Y** and **X → Z**

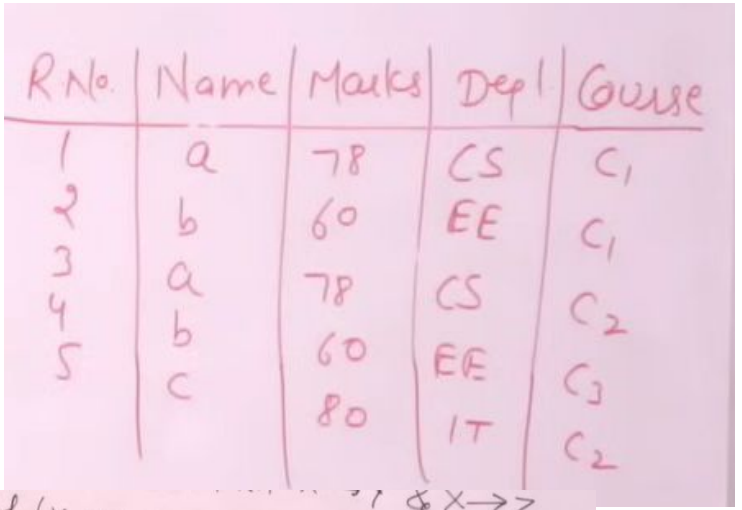
IR5. (Union/additive rule)

If **X → Y** and **X → Z**, then **X → YZ**

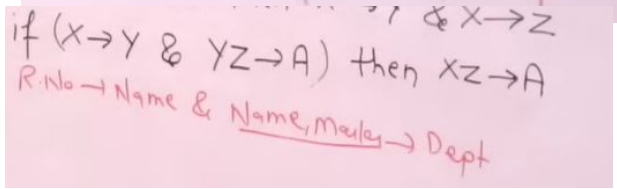


IR6. (Psuedotransitivity rule)

If **X → Y** and **WY → Z**, then **WX → Z**



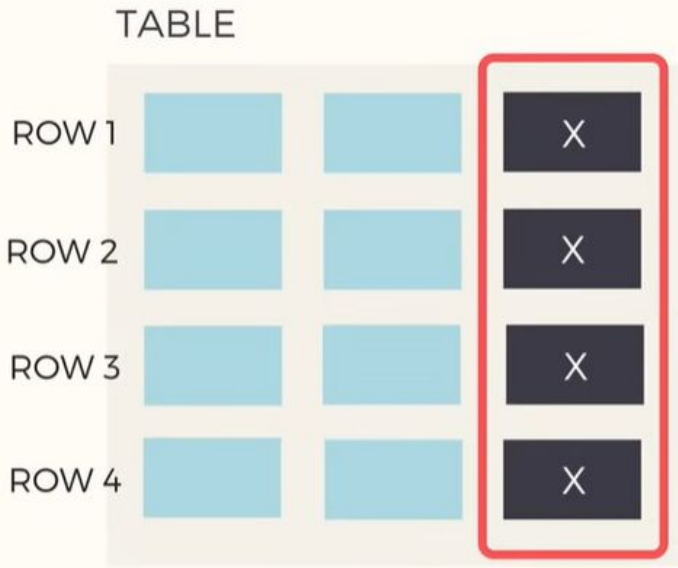
R No.	Name	Marks	Dept	Course
1	a	78	CS	C_1
2	b	60	EE	C_1
3	a	78	CS	C_2
4	b	60	EE	C_3
5	c	80	IT	C_2



Inference rules IR1 through IR3 are known as Armstrong's inference rules It has been **shown by Armstrong that inference rules IR1 through IR3 are sound and complete.**

normalization is

a technique of organizing the data into multiple related tables, to minimize DATA REDUNDANCY.

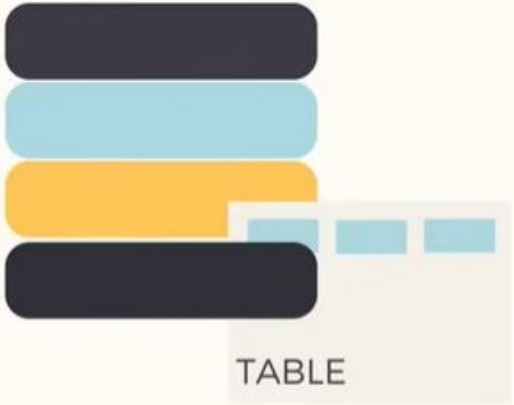


- Repetition of data increases the size of database.
- Other issues like:
  - Insertion Problems
  - Deletion Problems
  - Updation Problems

rollno	name	branch	hod	office_tel
1	Akon	CSE	Mr. X	53337
2	Bkon	CSE	Mr. X	53337
3	Ckon	CSE	Mr. X	53337
4	Dkon	CSE	Mr. X	53337

Unnecessary data repetition increases the size of the database.

And leads to more issues.



issues due to redundancy.

1. Insertion Anomaly
2. Deletion Anomaly
3. Updation Anomaly

Insertion Anomaly

To insert redundant data for every new row (of Student data in our case) is a data insertion problem or anomaly.

Deletion Anomaly

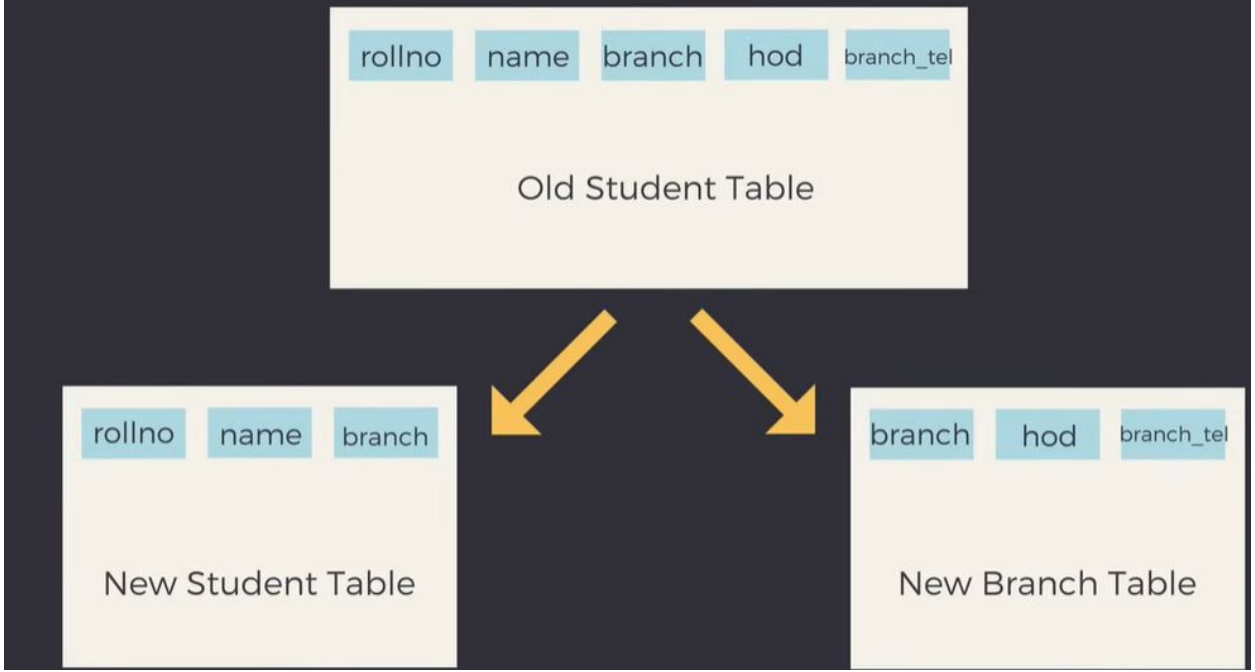
Loss of a related dataset when some other dataset is deleted.

STUDENTS TABLE

rollno	name	branch	hod	office_tel
1	Akon	CSE	Mr. X	53337
2	Bkon	CSE	Mr. X	53337
3	Ckon	CSE	Mr. X	53337
4	Dkon	CSE	Mr. X	53337

Mr. X leaves, and Mr. Y joins as the new HOD for CSE

How Normalization will solve all these problems?



STUDENTS TABLE

rollno	name	branch
1	Akon	CSE
2	Bkon	CSE
3	Ckon	CSE
4	Dkon	CSE

BRANCH TABLE

branch	hod	office_tel
CSE	Mr. Y	53337

Insertion problem solved.

STUDENTS TABLE

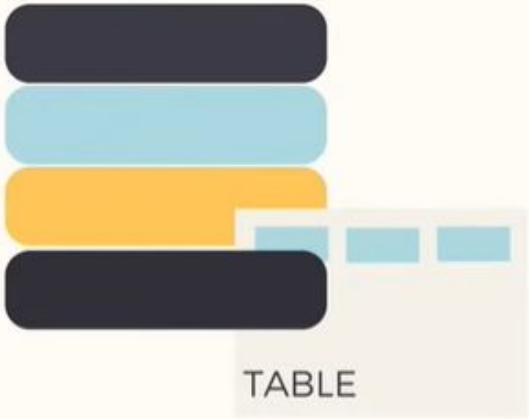
rollno	name	branch

No Data

BRANCH TABLE

branch	hod	office_tel
CSE	Mr. Y	53337

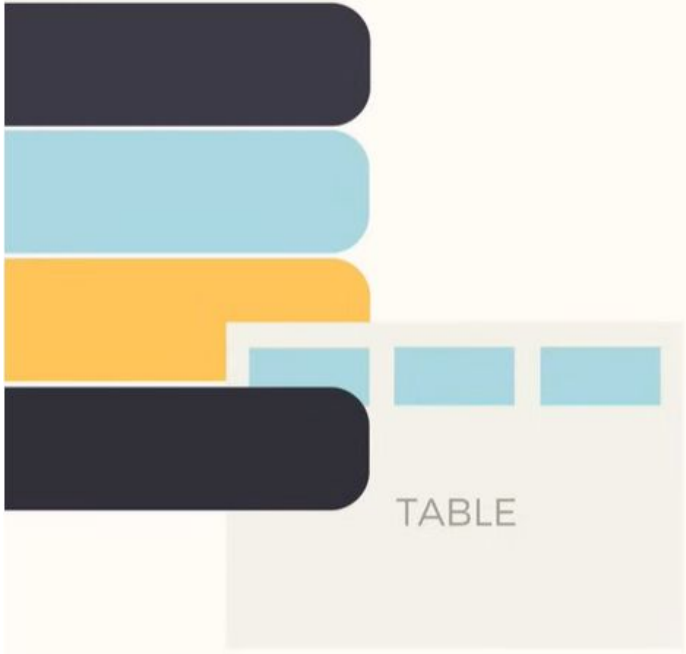
Deletion problem solved.



Types of Normalization

1. 1st Normal Form
2. 2nd Normal Form
3. 3rd Normal Form

1st Normal Form



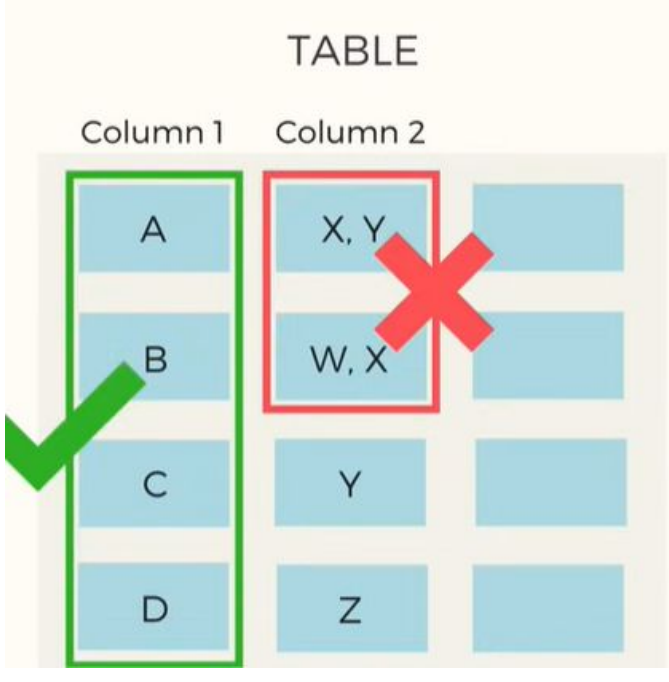
Step 1 of Normalisation process

- Scalable Table design which can be easily extended.
- If your table is not even in 1st Normal Form, it is considered poor DB design.

VERY BAD

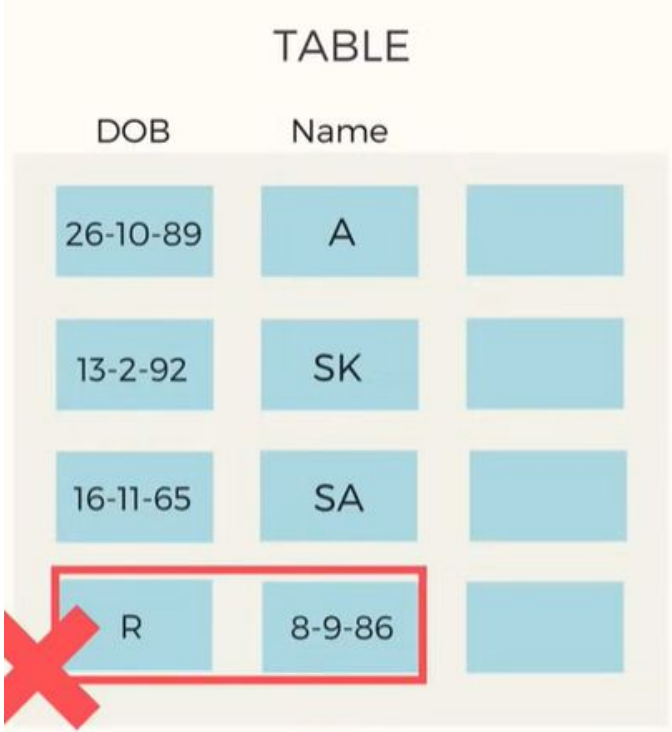
How to achieve 1st Normal Form?

There are 4 basic rules that a table should follow to be in 1st Normal Form.



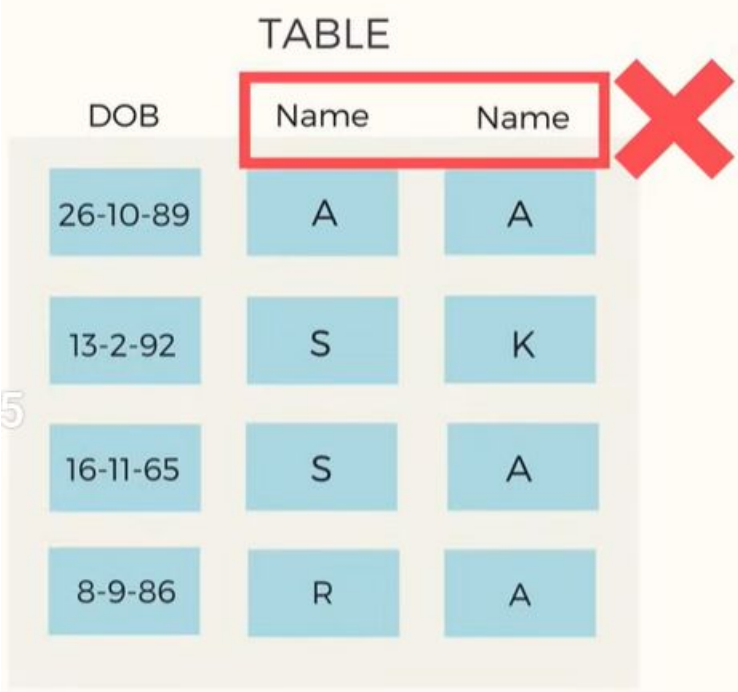
RULE 1

- Each Column should contain atomic values.
- Entries like X, Y and W, X violate this rule.



RULE 2

- A Column should contain values that are of the same type.
- Do not inter-mix different types of values in any column.



RULE 3

- Each column should have a unique name.
- Same names leads to confusion at the time of data retrieval

TABLE

Roll_no	F_Name	L_Name
3	A	A
4	S	K
1	S	A
2	R	A

RULE 4

- Order in which data is saved doesn't matter.
- Using SQL query, you can easily fetch data in any order from a table.

STUDENTS TABLE

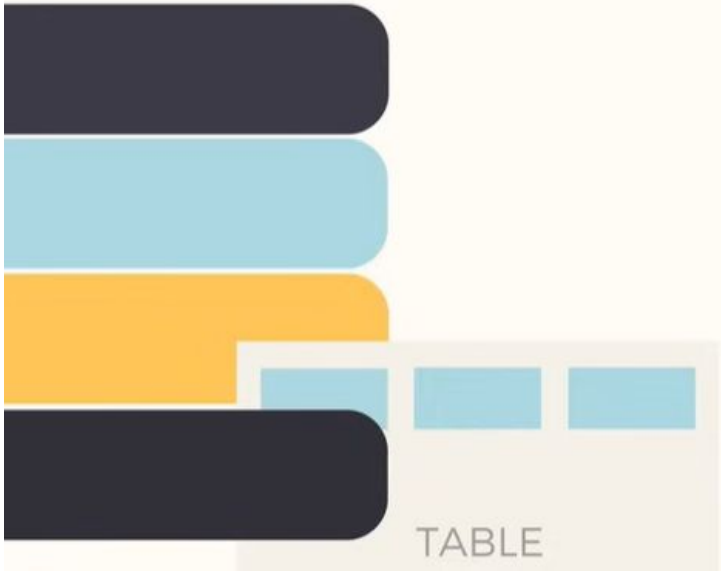
rollno	name	subject
101	Akon	OS, CN
103	Ckon	JAVA
102	Bkon	C, C++

Violation of 1 NF

STUDENTS TABLE

rollno	name	subject
101	Akon	OS
101	Akon	CN
103	Ckon	JAVA
102	Bkon	C
102	Bkon	C++

2nd Normal Form



For a table to be in the Second Normal Form...

- It should be in 1st Normal Form
- And, It should not have any Partial Dependencies.

To understand Partial Dependency, we first need to understand what **Dependency** is?

This is Dependency or Functional Dependency

STUDENTS TABLE

student_id	name	reg_no	branch	address
1	Akon	CSE-18	CSE	TN
2	Akon	IT-18	IT	AP
3	Bkon	CSE-18	CSE	HR
4	Ckon	CSE-18	CSE	MH

As the student_id in this table will be unique, it can be used easily to fetch any data.

SCORE TABLE

score_id	student_id	subject_id	marks	teacher
1	1	1	82	Mr. J
2	1	2	77	Mr. C++
3	2	1	85	Mr. J
4	2	2	82	Mr. C++
5	2	4	95	Mr. P

Primary Key should be score_id

But student_id + subject_id together makes a more meaningful primary key.

SCORE TABLE

score_id	student_id	subject_id	marks	teacher
1	1	1	82	Mr. J
2	1	2	77	Mr. C++
3	2	1	85	Mr. J
4	2	2	82	Mr. C++
5	2	4	95	Mr. P

in Score table

Primary key is a composition of two columns

score_id	student_id	subject_id	marks	teacher
1	10	1	82	Mr. J
2	10	2	77	Mr. C++
3	11	1	85	Mr. J
4	11	2	82	Mr. C++
5	11	4	95	Mr. P

teacher column only depends on subject and not on student.

This is Partial Dependency

And for 2nd Normal Form

A table should not have Partial Dependency.

SUBJECT TABLE

subject_id	subject_name	teacher
1	Java	Mr. J
2	C++	Mr. C++
3	C#	Mr. C#
4	Php	Mr. P

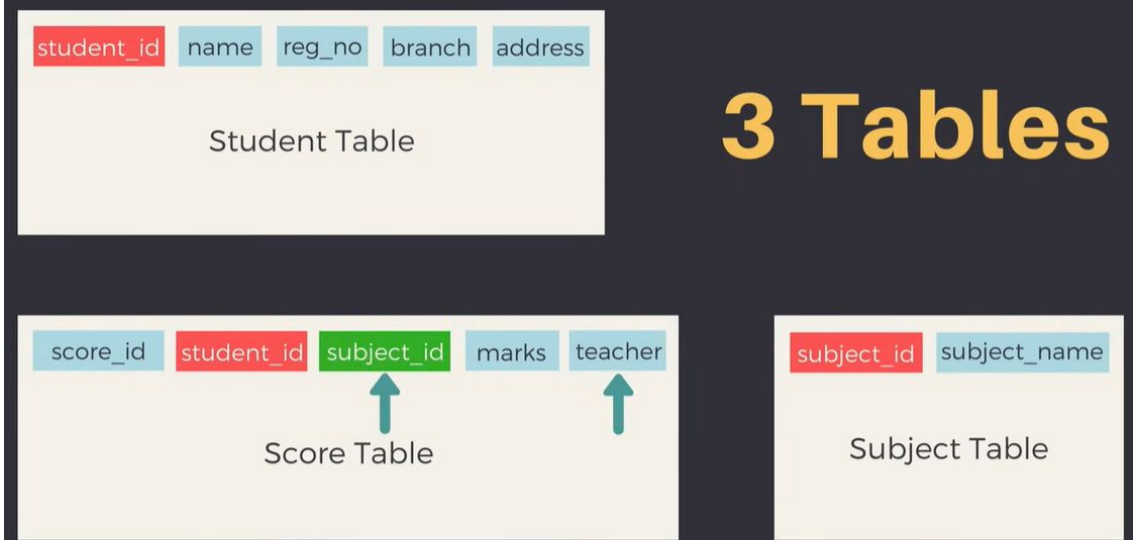
Makes more sense here...

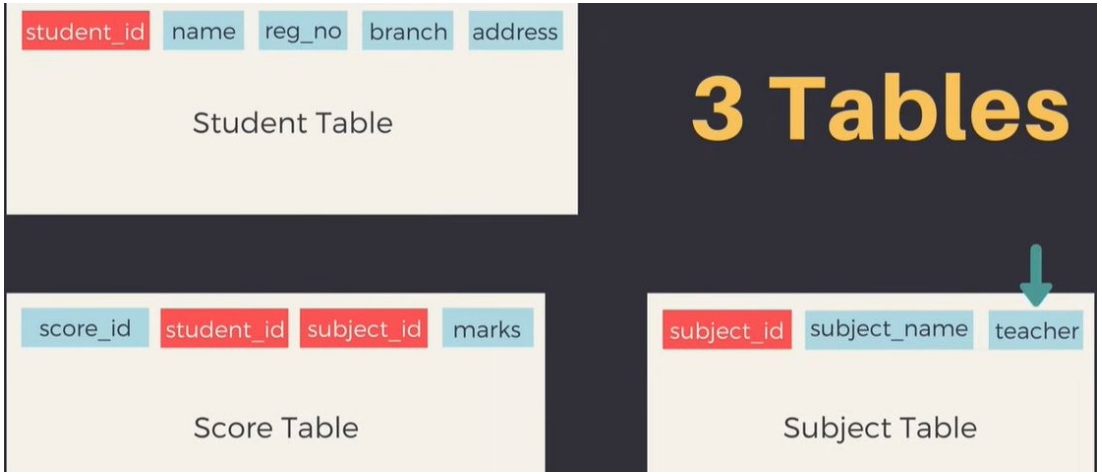
Our objective is to remove teacher column from Score table.

Teacher TABLE

teacher_id	teacher_name
1	Mr. J
2	Mr. C++
3	Mr. C#
4	Mr. P

Can even add more info. related to teachers like date of joining, salary etc.


student_id
name
reg_no
branch
address
Student Table
3 Tables
score_id
student_id
subject_id
marks
teacher
Score Table
subject_id
subject_name
Subject Table


student_id
name
reg_no
branch
address
Student Table
3 Tables
score_id
student_id
subject_id
marks
Score Table
subject_id
subject_name
teacher
Subject Table

For a table to be in 3rd Normal Form:

- It should be in 2nd Normal Form
- And it should not have Transitive Dependency.

SCORE TABLE

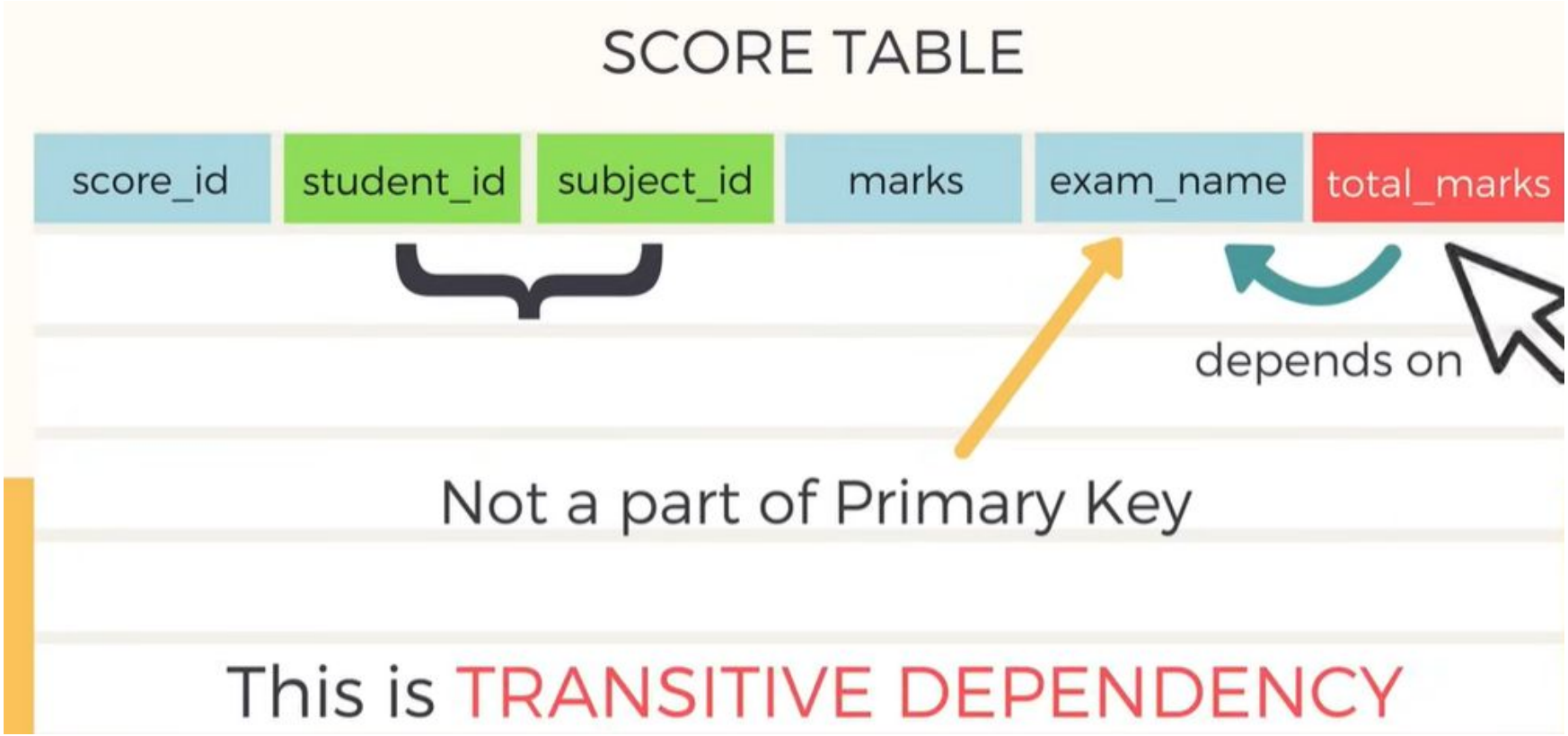
score_id	student_id	subject_id	marks	exam_name	total_marks

Primary Key for Score table is a Composite Key

total_marks depends on exam_name

Column exam_name depends on the primary key.

student_id + subject_id


SCORE TABLE
score_id
student_id
subject_id
marks
exam_name
total_marks
depends on
Not a part of Primary Key
This is TRANSITIVE DEPENDENCY

SCORE TABLE

score_id	student_id	subject_id	marks

And the solution to this problem is...

EXAM TABLE

exam_name	total_marks


student_id
name
reg_no
branch
address
Student Table
subject_id
subject_name
teacher
Subject Table
score_id
student_id
subject_id
marks
exam_name
Score Table
In 3rd
Normal Form
exam_name
total_marks
Exam Table

Different types of Keys

Keys:

- 1. Superkey:
- All possible combination of keys are called superkeys.
- Uniquely identify record.
- Superkey is a superset, we can derive other keys from superkey.

- {ID},
- {SSN},
- {ID,Name},{Name,SSN}
- {ID,SSN},{ID,phone},
- ID,email
- Name,email
- Name,ssn,email
- Id,SSN,Phone
- {Name,Salary}
- **Here, {Name,Salary} can't become a superkey.**

Employee

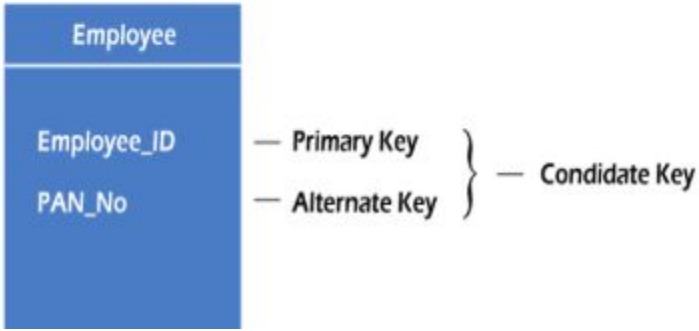
ID	Name	SSN	Salary	Phone	Email
101	John	AA	50000	12	j@sw
102	Robin	BB	60000	13	r@yh
103	Alya	CC	35000	14	a@hm
104	Yusuf	DD	68000	15	y@ch
105	John	EE	62000	89	j@in
106	Raj	FF	45000	87	r@au
107	Jayant	GG	25000	45	j@us
108	John	HH	35000	15	j@de
109	Neil	II	25000	12	n@uk

Candidate key

- Minimal set of superkey.
- From the following set of superkeys:
  {ID},{SSN},{ID,Name},{Name,SSN},{ID,SSN},{ID,phone},
  {ID,email},{Name,email},{Name,ssn,email},{Id,SSN,Phone},
  {Name,Salary}
- We can select candidate key (minimal set)
- Such as,
- {ID} and {SSN} can be a separate candidate keys.
- {email}, {Name,Phone}
- {ID,Name} cant be candidate key, bcz ID is a part of another candidate key.

Alternate Keys

The candidate key other than primary key are called alternate keys.



Unique key & Composite key

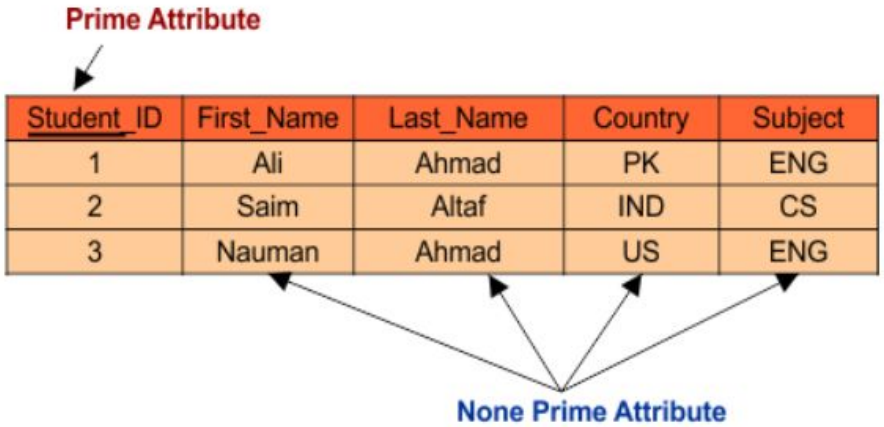
- It should contain unique values, but accept NULL values.
- Example:
- {Name,Phone} and {email} can be a unique keys.

- Composite key:
- Combination of Two or More attribute
- Example:
- {Name,Phone}
- {ID,Phone}
- {ID,Name}.. etc.

Non Prime Attribute in DBMS

In **DBMS** (database management system), the **non prime attribute** is characteristic of the table which cannot uniquely identify each tuple (row) in a table. Although **non prime attribute** is not helpful for a tuple identifier but it can provide detailed information about each tuple in that table.

Following is the descriptive diagram of non prime attribute



Example 1: Students Table

Let's consider a student table which contains the 5 attributes which includes Student-ID, Name, Age, subjects and GPA.

Student-ID	Name	Age	Subjects	GPA
1	Oliver	18	Computer Science	3.5
2	William	17	Mathematics	3.8
3	Anderson	22	English	3

- **Non Prime Attributes**: Name, Age, Subjects, GPA
- **Prime Attribute**: Student-ID

Example 2: Employees Table

Let's consider an Employees table which contains the 5 attributes which includes Employee -ID, Name, Department, Salary and Hire-Date.

Employee-ID	Name	Department	Salary	Hire-Date
1	Kane Williams	CS	70000	2024-01-10
2	Jone Hoda	HR	90000	2022-02-15
3	Anderson	English	110000	2021-02-15

- Non Prime Attributes: **Name, Department, Salary, Hire-Date**
- Primary Key (PK): **Employee-ID**

1. Not Part of Candidate Keys or Primary Key

Non Prime attribute will never the part of any candidate key of a table. Candidate Keys are used to uniquely identifies each row in a table

- **A candidate key** is a set of one or more attributes (columns) in a database table that can uniquely identify any record in the table without any redundancy. It is possible for a table to have more than one candidate key's.
- **However, a primary key** is a special candidate key selected by the database designer to uniquely identify every row in a table. primary will always be unique and not-null.

Example: Students Table

Student-ID	Roll-No	Age	Subject	GPA
1	Alice	20	Computer Science	3.5
2	Bob	22	Mathematics	3.7
3	Charlie	21	Physics	3.6
4	Diana	23	Chemistry	3.8

Explanation: In this table, **Student-ID** is the candidate key (primary key), while **Roll-No, Age, subject and CPA** are Non Prime attribute because they are not part of the candidate key.

Table R

A	B	C	D
SM	LQ	GT	HY
.	.	.	.
.	.	.	.
.	.	.	.

FUNCTIONAL DEPENDENCIES ::-

A, B ⟶ D

B ⟶ C

A, B ⟶ A, B, C, D

A, B TOGETHER IS A CANDIDATE KEY

$$\{ A , B \}^{+} = \{ A, B, C, D \}$$

Table R

A	B	C	D
SM	LQ	GT	HY
.	.	.	.
.	.	.	.
.	.	.	.

CANDIDATE KEY : { A , B }

PRIME ATTRIBUTE

A , B

NON PRIME ATTRIBUTE

C , D

PRIME ATTRIBUTE → NON PRIME ATTRIBUTE

CANDIDATE KEY : { A , B } - PRIMARY KEY

Table R

A	B	C	D
SM	LQ	GT	HY
.	.	.	.
.	.	.	.
.	.	.	.

CANDIDATE KEY :- {A, B}

B IS NOT A CANDIDATE KEY

PRIME ATTRIBUTE

A, B

NON PRIME ATTRIBUTE

C, D

Not a partial Dependent

DETERMINANT $\{A, B\}^+ = \{D\}$ DEPENDANT

COMPLETE CANDIDATE KEY

NON PRIME ATTRIBUTE

DETERMINANT $B^+ = \{C\}$ DEPENDANT

PART OF CANDIDATE KEY

NON PRIME ATTRIBUTE

PARTIAL DEPENDENCY

DEFINITION

Partial Dependency in DBMS occurs when a non-prime attribute depends on the proper subset or we can say part of the candidate key.

STEPS :-

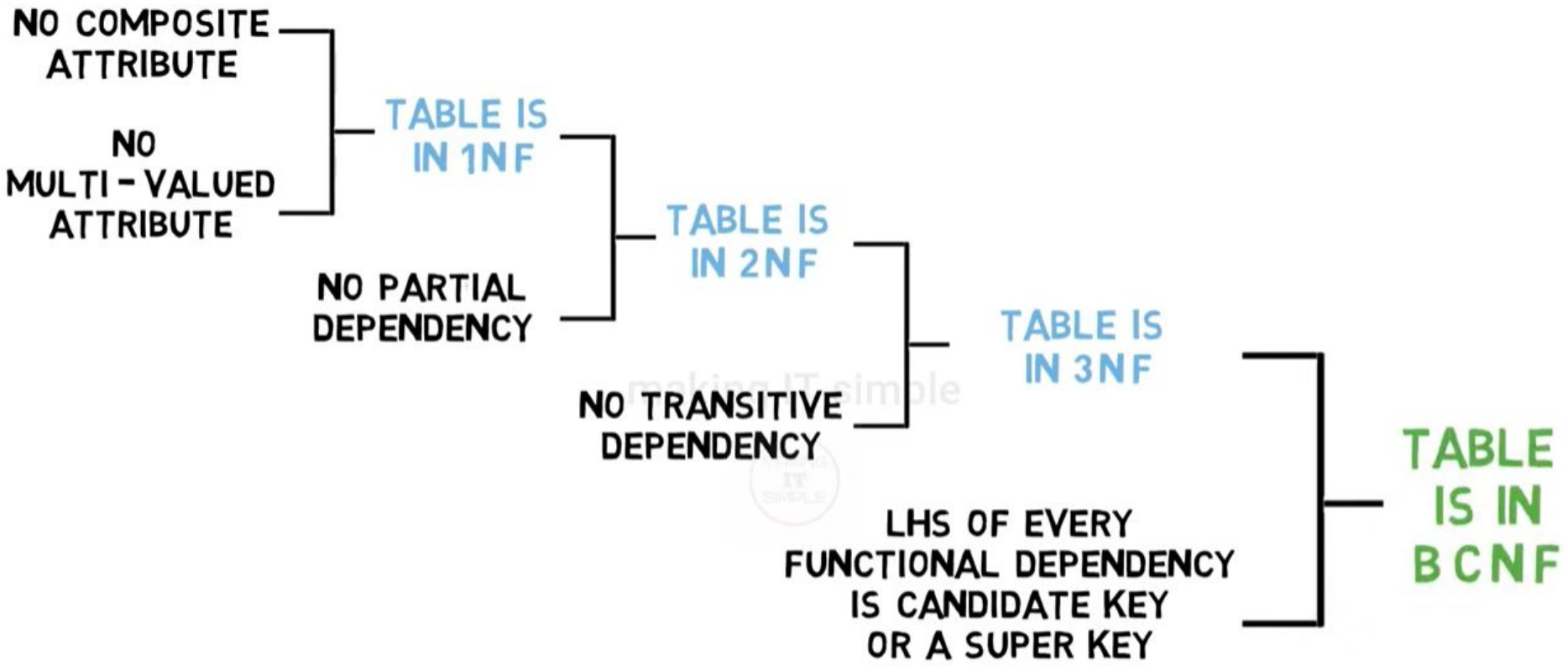
1) IDENTIFY CANDIDATE KEYS

2) DIFFERENTIATE PRIME AND NON-PRIME ATTRIBUTES

3) FIND A NON-PRIME ATTRIBUTE THAT IS DETERMINED BY A PART OR PROPER SUBSET OF CANDIDATE KEY

BCNF (Boyce-Codd Normal Form)

- A relation schema R is in **Boyce-Codd Normal Form (BCNF)** if whenever an **FD X -> A** holds in R, then **X is a superkey** of R
- In simple terms, for any case (say, X->Y), X can't be a non-prime attribute.
- Each normal form is strictly stronger than the previous one
  - Every 2NF relation is in 1NF
  - Every 3NF relation is in 2NF
  - Every BCNF relation is in 3NF
- There exist relations that are in 3NF but not in BCNF
- The goal is to have each relation in BCNF (or 3NF)



BOYCE - CODD NORMAL FORM

CONDITIONS FOR BOYCE - CODD NORMAL FORM

1) TABLE MUST BE IN THIRD NORMAL FORM (3NF)

2) EVERY FUNCTIONAL DEPENDENCY IN THE TABLE SHOULD BE OF THIS FORM

STRICTLY A CANDIDATE OR SUPER KEY

CONDITION 1: TABLE MUST BE IN THIRD NORMAL FORM (3NF)

Student	Subject	Teacher
ABC	Database	MNO
ABC	C	PQR
XYZ	Database	MNO
XYZ	C	STU

CANDIDATE KEY: { STUDENT, SUBJECT }

Student, Subject ⟶ Teacher

{ Student, Subject }$^{+}$ = { Student, Subject, Teacher }

FUNCTIONAL DEPENDENCIES:

Student, Subject ⟶ Teacher

Teacher ⟶ Subject

PARTIAL DEPENDENCY

X → Y

PART OF CANDIDATE KEY — NON-PRIME ATTRIBUTE

Student, Subject → Teacher

COMPLETE CANDIDATE KEY — NON-PRIME ATTRIBUTE

PARTIAL DEPENDENCY

(LHS) X → Y

PART OF CANDIDATE KEY — NON-PRIME ATTRIBUTE

Teacher → Subject (RHS)

NON-PRIME ATTRIBUTE — PART OF CANDIDATE KEY

Student	Subject	Teacher
ABC	Database	MNO
ABC	C	PQR
XYZ	Database	MNO
XYZ	C	STU

CANDIDATE KEY: { STUDENT, SUBJECT }

TRANSITIVE DEPENDENCY

X → Y

NON-PRIME ATTRIBUTE — NON-PRIME ATTRIBUTE

Student, Subject → Teacher

CANDIDATE KEY — NON-PRIME ATTRIBUTE

Teacher → Subject

NON-PRIME ATTRIBUTE — PART OF CANDIDATE KEY / PRIME ATTRIBUTE

1NF ✓
- NO COMPOSITE ATTRIBUTE
- NO MULTI-VALUED ATTRIBUTE

2NF ✓
- NO PARTIAL DEPENDENCY
- IS IN 1NF

3NF ✓
- NO TRANSITIVE DEPENDENCY
- IS IN 2NF

CONDITION 2: EVERY FUNCTIONAL DEPENDENCY IN THE TABLE IS OF FORM X DETERMINES Y WHERE X OR LHS IS STRICTLY A CANDIDATE KEY OR A SUPER KEY.

Student	Subject	Teacher
ABC	Database	MNO
ABC	C	PQR
XYZ	Database	MNO
XYZ	C	STU

Candidate Key: {Student, Subject}

Functional Dependencies:

Student, Subject → Teacher

Teacher → Subject

Teacher → Subject

NON-PRIME ATTRIBUTE

PART OF CANDIDATE KEY / PRIME ATTRIBUTE

(LHS) X → Y

CANDIDATE OR A SUPER KEY

Student, Subject → Teacher

CANDIDATE KEY

NON-PRIME ATTRIBUTE

R {STUDENT, SUBJECT, TEACHER}

R1
{STUDENT, TEACHER}
FK

R2
{TEACHER, SUBJECT
PK

BOYCE - CODD NORMAL FORM

4th Normal Form

- It should satisfy BCNF
- It should not have Multi-Valued Dependency.

In 2NF we removed Partial Dependency.

and, In 3NF we removed Transitive Dependency.

For a table with A, B, C columns

$A \rightarrow B$, is Multi-Valued Dependency

Then B and C should be independent of each other.

3 conditions for Multivalued Dependency

1. A□B ,for a single value of A, more than one value of B exists.
2. Tables should have at least 3 columns.

(if table has only 2 columns, we can use 1NF to resolve it).

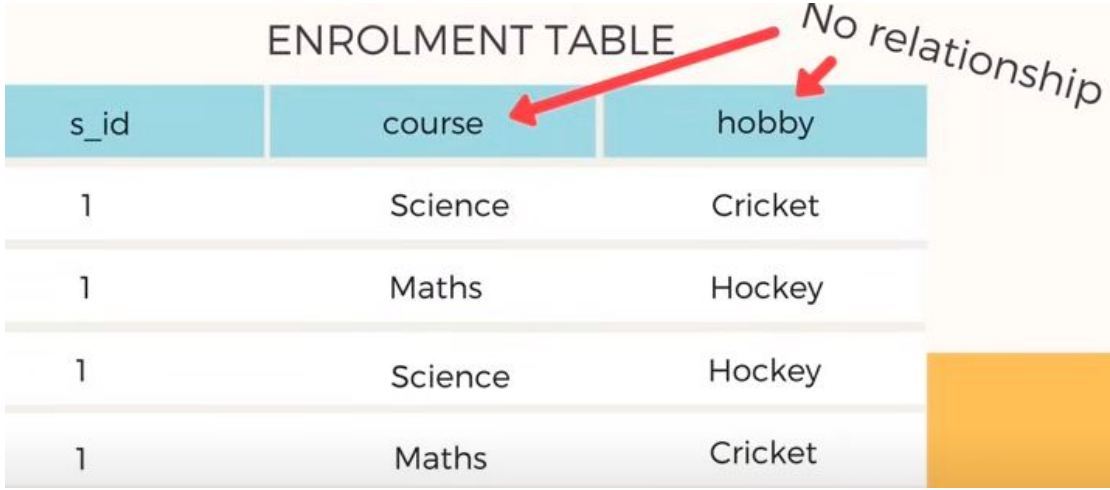
3.For this table with A,B,C columns, B and C should be independent.

If ALL THE 3 CONDITIONS ARE TRUE, THEN WE CAN SAY THAT THE TABLE MAY HAVE MULTI-VALUED DEPENDENCY.

EXAMPLE 1

ENROLMENT TABLE

s_id	course	hobby
1	Science	Cricket
1	Maths	Hockey



- DECOMPOSITION of the table:

CourseOpted TABLE

s_id	course
1	Science
1	Maths
2	C#
2	Php

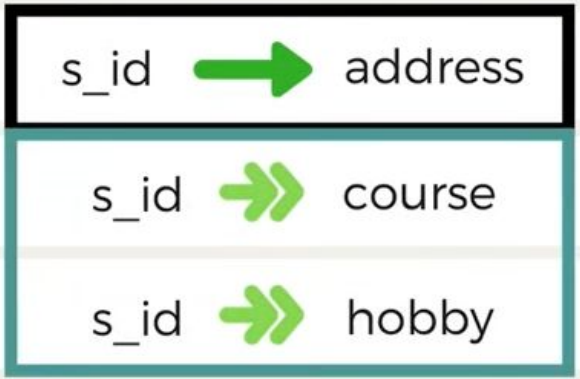
Hobbies TABLE

s_id	hobby
1	Cricket
1	Hockey
2	Cricket
2	Hockey

2 independent tables

ENROLMENT TABLE

s_id	address	course	hobby



Functional Dependency

Multi-valued Dependency

Student Enrollment Table

CourseOpted Table + **Hobbies Table** + **Address Table**

(s_id & course) (s_id & hobby) (s_id & address)

Multi-valued Dependency occurs due to bad database design.

Student Table

s_id	s_name
S1	A
S2	B

Course Table

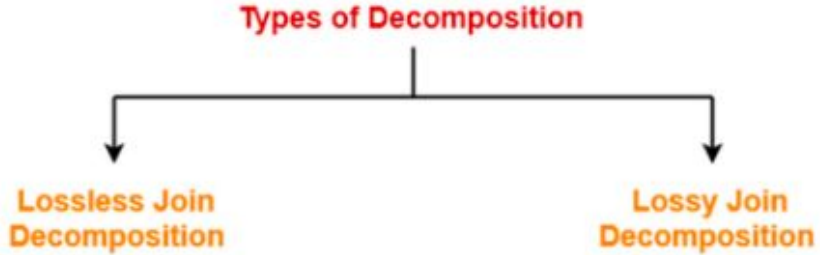
c_id	c_name
C1	C
C2	D

Good Database Design

Decomposition

- The process of decomposition in DBMS helps us remove
  - redundancy,
  - Inconsistencies,
  - anomalies from a database when we divide the table into numerous tables.
- In simpler words, the process of decomposition refers to dividing a relation X into {X1, X2,……Xn}.

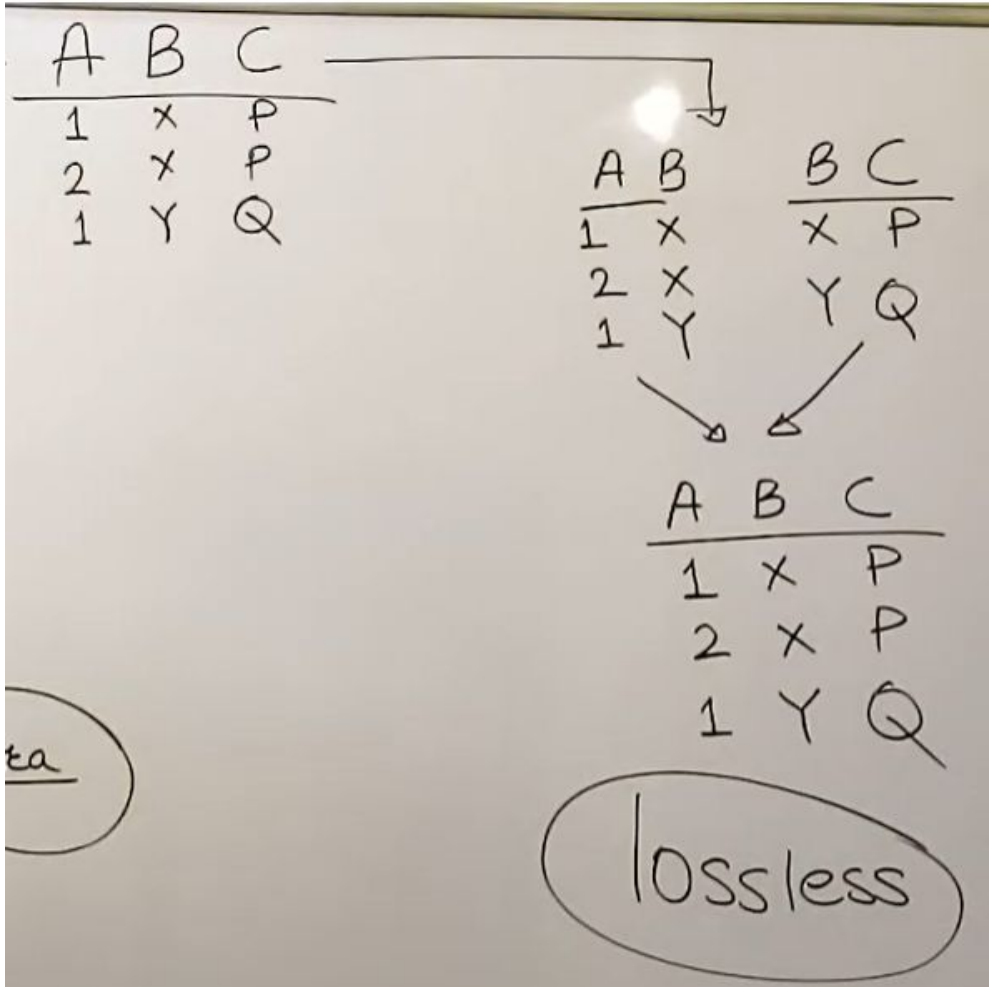
Types of Decomposition



- **Lossless Join Decomposition (Non-additive):**
- Consider there is a relation R which is decomposed into sub relations R1 , R2 , …. , Rn.
- This decomposition is called lossless join decomposition when the join of the sub relations results in the same relation R that was decomposed.
- For lossless join decomposition, we always have

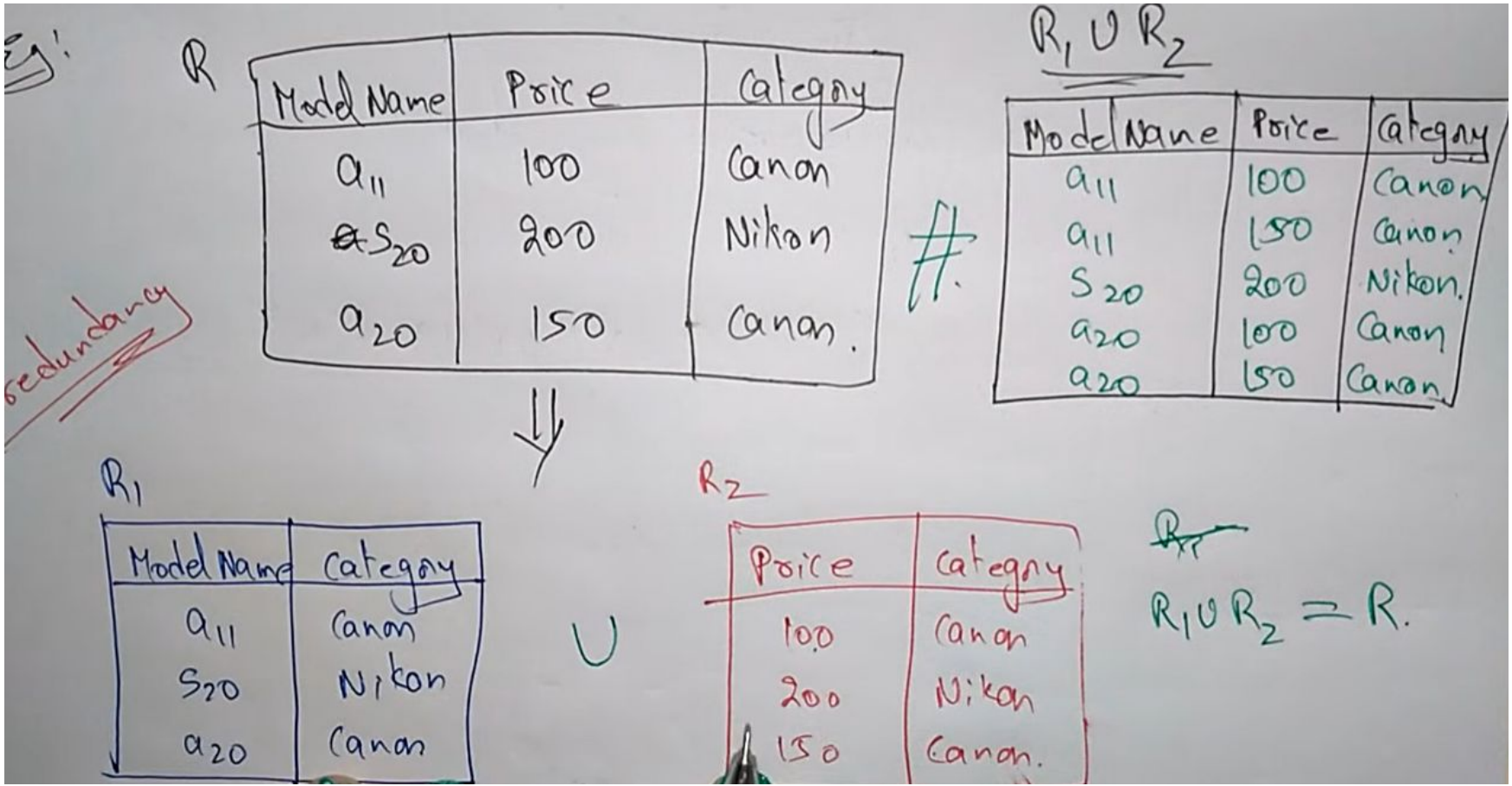
  R1 ⋈ R2 ⋈ R3 ……. ⋈ Rn = R, where ⋈ is a natural join operator

Lossless Decomposition



- **Lossy Join Decomposition:**
- Consider there is a relation R which is decomposed into sub relations R1 , R2 , …. , Rn.
- This decomposition is called lossy join decomposition when the join of the sub relations does not result in the same relation R that was decomposed.
- The natural join of the sub relations is always found to have some extraneous tuples.
- For lossy join decomposition, we always have-
- $R1 \bowtie R2 \bowtie R3 \ldots\ldots \bowtie Rn \supset R$, where $\bowtie$ is a natural join operator

Lossy Decomposition



Determining Whether Decomposition Is Lossless Or Lossy

- Consider a relation R is decomposed into two sub relations R1 and R2. Then,
- If all the following conditions satisfy, then the decomposition is lossless.
- If any of these conditions fail, then the decomposition is lossy.

- **Condition-01**
- Union of both the sub relations must contain all the attributes that are present in the original relation R.
- Thus, R1 ∪ R2 = R

Cntd...

- **Condition-02**
- Intersection of both the sub relations must not be null.
- In other words, there must be some common attribute which is present in both the sub relations.
- Thus, $R1 \cap R2 \neq \emptyset$
- **Condition-03**
- Intersection of both the sub relations must be a super key of either R1 or R2 or both.
- Thus, $R1 \cap R2$ = Super key of R1 or R2

Properties of Decomposition

- **Lossless:**
- All the decomposition that we perform in Database management system **should be lossless.**
- All the information should not be lost while performing the join on the sub-relation to get back the original relation. It helps to remove the redundant data from the database.
- **Dependency Preservation:**
- Dependency Preservation is an important technique in database management system.
- **It ensures that the functional dependencies between the entities is maintained while performing decomposition.**
- It helps to improve the database efficiency, maintain consistency and integrity.
- **Lack of Data Redundancy:**
- Data Redundancy is generally termed as duplicate data or repeated data.
- This property states that the decomposition performed **should not suffer redundant data.**
- It will help us to get rid of unwanted data and focus only on the useful data or information.

Dependency preservation example